

Custom QEMU ARM virt platform

Page Content

- [General](#)
- [Known issues for seL4](#)
 - [Lack of UARTs](#)
 - [Lack of timer peripherals](#)
- [Building a custom QEMU](#)
 - [Utilization of a custom qemu-arm-virt platform](#)
 - [Build QEMU](#)
- [Using a custom QEMU within the TRENTOS SDK](#)
- [Using a custom QEMU within seL4 projects](#)
 - [Adding timer driver support in seL4](#)
 - [seL4](#)
 - [util_libs](#)
 - [global-components/TimeServer](#)
 - [Adding UART driver support in seL4](#)
 - [Build and run seL4Test](#)
 - [Build and run the CAMkES VM examples](#)
 - [Building own Linux Kernel](#)

General

See also [QEMU](#) for a general background about QEMU.

Known issues for seL4

Lack of UARTs

QEMU provides just one UART, which is used by seL4 kernel debug logging. There are no UARTs for userspace applications.

Lack of timer peripherals

The seL4 kernel does not provide a timer API for userspace. It expects the hardware to have a timer peripheral that a userspace timer driver/server can use to provide such a service. Technically, the seL4 kernel could provide a timer API, but it was considered to violate the microkernel principles. The MCS version supports certain use cases and there is a discussion to provide a userspace timer API in non-MCS for conveniences reasons.

The default QEMU ARM virt platform in mainline QEMU lacks support for a dedicated timer peripheral which could be used by seL4 as a user-space timer. Currently, two options exist to extend seL4 support - one minimally invasive using the RTC, the other requiring a modification to the QEMU platform itself to add a timer. The default QEMU ARM virt platform provides support for an [ARM PrimeCell Real Time Clock \(PL031\)](#) device, which can be used as a regular RTC but also allows for configuring an alarm. It could therefore be used as a regular timer device when using an adequate device driver. Within **libplatsupport** in **util_libs**, a corresponding timer driver (both timer and ltimer) was added as part of

[SEOS-3338](#) - Getting issue details...

STATUS

. Support was also extended on the CAMkES level as part of the TRENTOS **TimeServer**

component (see [SEOS-3341](#) - Getting issue details...

STATUS

). Unfortunately, tests utilizing the sel4test suite showed that the resolution of the PL031 RTC device (only in seconds) is too low compared to the resolution (in milliseconds) required by typical seL4 components (e.g. the CAMkES ARM VMM). Therefore, the timer based on the PL031 RTC device can't be used adequately within the existing virtualization examples.

Besides diverting an already existing device like the PL031 RTC, one could also extend the underlying QEMU ARM virt platform in order to provide dedicated timer hardware support like it is available on regular physical and emulated platforms. The [ARM Dual-Timer Module \(SP804\)](#) hereby marks a device that is already very well supported in QEMU. The following guide provides the steps for reproducing the respective setup.

Building a custom QEMU

See <https://wiki.qemu.org/Hosts/Linux> and https://en.wikibooks.org/wiki/QEMU/Installing_QEMU for general information how to setup the build environment. The following packages must be installed

```
$ sudo apt install build-essential bison flex python pkg-config libglib-perl zlib1g-dev libglib2.0-dev libpixman-1-dev libpixman-1-dev
```

An archive with all sources of a released QEMU version can be downloaded from <https://download.qemu.org>. For tracking of changes, upstreaming or using a cutting-edge version, nevertheless the QEMU GitLab repository should be cloned:

```
$ git clone https://gitlab.com/qemu-project/qemu.git
$ cd qemu
$ git submodule update --init --recursive --jobs 8
$ git checkout v8.0.0-rc1
```

Utilization of a custom qemu-arm-virt platform

The default qemu-arm-virt platform lacks support of user-space timers and additional UARTs. Unfortunately, a timer device is required for the execution of specific seL4 VM examples (e.g. `vm_serial_server`). A modified version that already contains

- 2x ARM SP804 dual timer
- 5x ARM PL011 UART

can thus be found at https://gitlab.com/s0ckl/qemu/-/tree/custom_qemu-arm-virt?ref_type=heads. This version can be checked out and used in combination with both mainline seL4 VM examples and the TRENTOS SDK.

Build QEMU

The parameter "`--target-list=<arch1>[,<arch2>[...]]`" defines what architectures QEMUs will be created for. Certain features should be disabled to reduce unnecessary host dependencies, so QEMU runs within the seL4/CAMkES docker based build environment

```
$ mkdir build
$ cd build
$ ../configure --target-list=aarch64-softmmu --disable-snappy --disable-vnc --disable-vnc-jpeg --disable-vnc-sasl --disable-sdl --disable-pa --disable-brlapi --disable-gtk --disable-libiscsi --disable-libnfs --disable-rbd --disable-libusb
$ make -j8
```

Using a custom QEMU within the TRENTOS SDK

The branch `SEOS-000_custom_qemu_arm_virt` within `seos_tests` already contains all required modifications for executing the above mentioned modified QEMU version in combination with the TRENTOS SDK. For being able to access the custom QEMU from within the TRENTOS docker based build and test environment, we nevertheless have to copy the QEMU *build* folder into the `seos_tests` root folder.

Using a custom QEMU within seL4 projects

 **This describes things when using the seL4 build system, not the TRENTOS SDK and the test framework.** 

Before being able to add the required timer support in seL4, we first have to checkout the respective project with the help of the according manifest. As a first step, it is recommended to test the timer device implementation with the help of a timer driver for the SP804 device that is already provided by seL4. The seL4test suite provides a solid foundation for that. In a second step, the new timer driver can be used to extend the existing CAMkES VM examples for the QEMU ARM virt platform. Currently, the following two projects are supported:

- [sel4test-manifest](#)
- [camkes-vm-examples-manifest](#)

For being able to access the custom QEMU from within the seL4/CAMkES docker based build environment, we now have to copy the QEMU *build* folder into the respective seL4 project's own *build* folder, e.g. having a structure like `../sel4test/build/build` or `../camkes-vm-examples/build/build`.

Within the simulation CMake script, we also have to provide the modified path to our custom QEMU build and add the command line parameter "`-nic none`", in order to avoid the missing efi-rom error on ARM system:

```
seL4_tools/cmake-tool/helpers/simulation.cmake

...
elseif(KernelPlatformQEMUArmVirt)
    set(QemuBinaryMachine "/host/build/build/qemu-system-${QEMU_ARCH} -nic none")
...

```

Adding timer driver support in seL4

Regardless of which project was checked out before, the following changes have to be done within the respective projects subfolders

- `../sel4test/projects/`
- `../sel4test/kernel`
- `../sel4test/tools`

in order to equip the sel4 environment for support of the SP804 device.

sel4

Within the kernel config CMake script, we have to provide the modified path to our custom QEMU build and add the command line parameter `-nic none`, in order to avoid the missing efi-rom error:

seL4/src/plat/qemu-arm-virt/config.cmake

```
...
    # Use the system's QEMU if no custom QEMU is provided. Have a sanity
    # check about the version to ensure it can be used.
    if(NOT QEMU_BINARY)
        set(QEMU_BINARY "/host/build/build/qemu-system-${QEMU_ARCH}")
    ...
    ...

    # Run QEMU to get the device tree binary. Remember the command, so
    # it can be added to the DTS as reference. Every parameter must be
    # a separate string, as CMake will make it a dedicated argument
    # passed to QEMU then (e.g. "-machine virt" wont be recognized,
    # but "-machine", "vir" is).
    set(
        QEMU_CMD
        "${QEMU_BINARY}"
        "-machine"
        "${QEMU_MACHINE}"
        "-cpu"
        "${ARM_CPU}"
        "-smp"
        "${QEMU_SMP_OPTION}"
        "-m"
        "${QEMU_MEMORY}"
        "-nographic"
        "-nic"
        "none"
    )
...
```

We also have to adapt the maximum number of IRQs to 256, in order to allow sel4 to handle our newly added timer IRQs in QEMU.

seL4/src/plat/qemu-arm-virt/config.cmake

```
...
declare_default_headers(
    TIMER_FREQUENCY 62500000
    MAX_IRQ 256
    NUM_PPI 32
    TIMER drivers/timer/arm_generic.h
    INTERRUPT_CONTROLLER arch/machine/gic_v2.h
    CLK_MAGIC 4611686019llu
    CLK_SHIFT 58u
    KERNEL_WCET 10u
)
...
```

util_libs

We first have to add the respective timer drivers within `libplatsupport` in the `util_libs` folder, consisting of both timer and ltimer. Luckily, sel4 already provides support for the SP804 device as part of the FVP platform. We can therefore just copy the respective files

- `/util_libs/libplatsupport/plat_include/fvp/platsupport/plat/sp804.h` to `/util_libs/libplatsupport/plat_include/qemu-arm-virt/platsupport/plat/`
- `/util_libs/libplatsupport/src/plat/fvp/sp804.c` to `/util_libs/libplatsupport/src/plat/qemu-arm-virt/`
- `/util_libs/libplatsupport/src/plat/fvp/ltimer.c` to `/util_libs/libplatsupport/src/plat/qemu-arm-virt/`

We now must adapt **sp804.h**, in order to provide the respective DTB paths:

util_libs/libplatsupport/plat_include/qemu-arm-virt/platsupport/plat/sp804.h

```
...
#define SP804_TIMER1_PATH "/sp804@90c0000"
#define SP804_TIMER2_PATH "/sp804@90d0000"
...
```

We also have to adapt **sp804.c**, in order to provide the correct clock frequency:

util_libs/libplatsupport/src/plat/qemu-arm-virt/sp804.c

```
...
#define TICKS_PER_SECOND 1000000
...
```

global-components/TimeServer

In order to run CAMkES VM examples which require timer support (e.g. **vm_serial_server**), the TimeServer component has to be adapted as well by modifying the existing file (e.g. `/global-components/components/TimeServer/include/plat/qemu-arm-virt/plat/timers.h`).

global-components/components/TimeServer/include/plat/qemu-arm-virt/plat/timers.h

```
/*
 * Copyright 2019, Data61
 * Commonwealth Scientific and Industrial Research Organisation (CSIRO)
 * ABN 41 687 119 230.
 *
 * This software may be distributed and modified according to the terms of
 * the BSD 2-Clause license. Note that NO WARRANTY is provided.
 * See "LICENSE_BSD2.txt" for details.
 *
 * @TAG(DATA61_BSD)
 */
#pragma once

#define HARDWARE_TIMER_INTERFACES
    emits Dummy dummy_source;
    consumes Dummy timer0;
    consumes Dummy timer1;
#define HARDWARE_TIMER_ATTRIBUTES
#define HARDWARE_TIMER_COMPOSITION
    connection sel4DTBHardware timer0_conn(from dummy_source, to timer0);
    connection sel4DTBHardware timer1_conn(from dummy_source, to timer1);
#define HARDWARE_TIMER_CONFIG
    timer0.dtb = dtb({"path" : "/sp804@90c0000"});
    timer0.generate_interrupts = 1;
    timer1.dtb = dtb({"path" : "/sp804@90d0000"});
    timer1.generate_interrupts = 1;
#define HARDWARE_TIMER_PLAT_INTERFACES
```

Adding UART driver support in seL4

TODO

Build and run seL4Test

In order to checkout the seL4Test suite, one can follow the steps listed below. Further information can also be found [here](#).

```
$ mkdir sel4test
$ cd sel4test
$ repo init -u https://github.com/sel4/sel4test-manifest.git
$ repo sync
$ container
$ mkdir build
$ cd build
```

Within sel4test, we then also allow the timer related tests to be executed on the QEMU ARM virt platform by temporarily modifying the config parameter **SeL4testHaveTimer** from OFF to ON.

```
../projects/sel4test/settings.cmake
```

```
...
else()
    set(Sel4testHaveTimer ON CACHE BOOL "" FORCE)
endif()
...
```

Run sel4test

```
$ ../init-build.sh -DPLATFORM=qemu-arm-virt -DSIMULATION=TRUE
$ ninja
$ ./simulate
```

Build and run the CAMkES VM examples

In order to checkout the CAMkES VM environment and respective VM examples, one can follow the steps listed below. Further information can also be found [here](#).

```
$ mkdir camkes-vm-examples
$ cd camkes-vm-examples
$ repo init -u https://github.com/sel4/camkes-vm-examples-manifest
$ repo sync
$ container
$ mkdir build
$ cd build
```

In order to add a new platform to one of the available CAMkES ARM VM examples (e.g. **vm_serial_server**), one has to provide/modify the following files:

- **<plat_folder>/devices.camkes**
- **CMakeLists.txt**
- **settings.cmake**

The following files exemplarily address the **vm_serial_server** example, where we add the new QEMU ARM virt platform. As a first step, we have to add a new file **devices.camkes**, which describes our platform, respective dependencies/restrictions (**linux_address_config**, **linux_image_config**) and also defines the devices that shall be passthrough to a respective Linux VM (**dtb**, **untyped_mmios**).

```
camkes-vm-examples/apps/Arm/vm_serial_server/qemu-arm-virt/devices.camkes
```

```
/*
 * Copyright (C) 2023, HENSOLDT Cyber GmbH
 *
 * SPDX-License-Identifier: BSD-2-Clause
 */

#include <configurations/vm.h>

#define VM_INITRD_MAX_SIZE 0x19000000 //25 MB
#define VM_RAM_BASE 0x40000000
#define VM_RAM_SIZE 0x20000000
#define VM_RAM_OFFSET 0x00000000
```

```

#define VM_DTB_ADDR 0x4F000000
#define VM_INITRD_ADDR 0x4D700000

assembly {
    composition {}
    configuration {

        vm.linux_address_config = {
            "linux_ram_base" : VAR_STRINGIZE(VM_RAM_BASE),
            "linux_ram_paddr_base" : VAR_STRINGIZE(VM_RAM_BASE),
            "linux_ram_size" : VAR_STRINGIZE(VM_RAM_SIZE),
            "linux_ram_offset" : VAR_STRINGIZE(VM_RAM_OFFSET),
            "dtb_addr" : VAR_STRINGIZE(VM_DTB_ADDR),
            "initrd_max_size" : VAR_STRINGIZE(VM_INITRD_MAX_SIZE),
            "initrd_addr" : VAR_STRINGIZE(VM_INITRD_ADDR)
        };

        vm.linux_image_config = {
            "linux_bootcmdline" : "console=hvc0 nosmp rw debug loglevel=8 pci=noms,realloc=off,bios
initcall_blacklist=clk_disable_unused",
            "linux_stdout" : "hvc0",
        };

        vm.dtb = dtb([{}]);

        vm.untyped_mmios = [
            "0x8020000:12",
            "0x8040000:12", // Interrupt Controller Virtual CPU interface (Virtual Machine view)
            "0x40000000:29", // Linux kernel memory regions
        ];
    }
}

```

Afterwards, we have to configure the build process by modifying the existing **CMakeLists.txt** file.

camkes-vm-examples/apps/Arm/vm_serial_server/CMakeLists.txt

```

#
# Copyright (C) 2023, HENSOLDT Cyber GmbH
#
# SPDX-License-Identifier: BSD-2-Clause
#

cmake_minimum_required(VERSION 3.8.2)

project(camkes-vm-serial-server C)

include(${CAMKES_ARM_VM_HELPERS_PATH})

find_package(camkes-vm-linux REQUIRED)
include(${CAMKES_VM_LINUX_HELPERS_PATH})

if("${KernelARMPlatform}" STREQUAL "exynos5422")
    # Create our CPP Flags based on ARM VM config variables
    set(cpp_flags "-DKERNELARMPLATFORM_EXYNOS5422")
elseif("${KernelARMPlatform}" STREQUAL "qemu-arm-virt")
    # Create our CPP Flags based on ARM VM config variables
    set(cpp_flags "-DKERNELARMPLATFORM_QEMU-ARM-VIRT")
    set(SIMULATION ON CACHE BOOL "Generate simulation script to run qemu with the proper arguments")
    include(simulation)
    if(SIMULATION)
        GenerateSimulateScript()
    endif()
endif()

# Generate overlayed rootfs
set(rootfs_file "${CAMKES_VM_IMAGES_DIR}/${KernelARMPlatform}/rootfs.cpio.gz")

```

```

# Setup our passthrough vm overlay
AddFileToOverlayDir(
    "inittab"
    ${CMAKE_CURRENT_SOURCE_DIR}/overlay_files/init_scripts/inittab_hvc0
    "etc"
    overlay
)

AddOverlayDirToRootfs(
    overlay
    ${rootfs_file}
    "buildroot"
    "rootfs_install"
    output_overlayed_rootfs_location
    rootfs_target
    GZIP
)

AddToFileServer(
    "linux-initrd"
    ${output_overlayed_rootfs_location}
    DEPENDS
    rootfs_target
)

AddToFileServer("linux" "${CAMKES_VM_IMAGES_DIR}/${KernelARMPlatform}/linux")

AddCamkesCPPFlag(cpp_flags CONFIG_VARS VmEmmc2NoDMA VmUSB Tk1DeviceFwd Tk1Insecure)

DefineCAMkESVMFileServer()

CAMkESAddImportPath(${KernelARMPlatform})

# Declare root server
DeclareCAMkESRootserver(
    vm_serial_server.camkes
    CPP_FLAGS
    ${cpp_flags}
    CPP_INCLUDES
    ${CAMKES_VM_DIR}/components/VM_Arm
)

```

And in a last step we also have to adapt the existing **settings.cmake** file.

camkes-vm-examples/projects/vm-examples/apps/Arm/vm_serial_server/settings.cmake

```

#
# Copyright (C) 2023, HENSOLDT Cyber GmbH
#
# SPDX-License-Identifier: BSD-2-Clause
#

set(supported "exynos5422;qemu-arm-virt")
if(NOT "${PLATFORM}" IN_LIST supported)
    message(FATAL_ERROR "PLATFORM: ${PLATFORM} not supported.
        Supported: ${supported}")
endif()
set(KernelARMPlatform "${PLATFORM}" CACHE STRING "" FORCE)
set(LibUSB OFF CACHE BOOL "" FORCE)
set(VmPCISupport ON CACHE BOOL "" FORCE)
set(VmVirtioConsole ON CACHE BOOL "" FORCE)
set(VmInitRdFile ON CACHE BOOL "" FORCE)

if(${PLATFORM} STREQUAL "exynos5422")
endif()
if(${PLATFORM} STREQUAL "qemu-arm-virt")
    # force cpu

```

```
set(QEMU_MEMORY "2048")
set(KernelArmCPU cortex-a53 CACHE STRING "" FORCE)
endif()
```

Run the VM example

```
$ ../init-build.sh -DCAMKES_VM_APP=vm_serial_server -DPLATFORM=qemu-arm-virt
$ ninja
$ ./simulate
```

Building own Linux Kernel

The currently provided linux kernel is version 4.19. If you want to replace the kernel you can use this build script <https://bitbucket.hensoldt-cyber.systems/projects/SSDK/repos/dev-scripts/browse/build-linux-kernel.sh?at=refs%2Fheads%2Fvm-examples-wp0>. To compile the linux kernel for arm64/aarch64 on Ubuntu you need to install the following packages:

```
sudo apt install binutils-aarch64-linux-gnu gcc-aarch64-linux-gnu
```

More information can be found under <https://gts3.org/2017/cross-kernel.html>

The build script is called:

```
./build-linux-kernel.sh 5 18.1
```

where the first argument 5 means we want to build linux major version 5 and the second argument 18.1 that we want minor version 18.1. This split is needed to find the correct http path for the kernel source. The final kernel will be copied back into the start directory with the naming scheme linux-arm64-5.18.1.

If changes need to be made to the configuration you can perform them manually, simply pick up the script the outcommented at make menuconfig. menuconfig will open up a menu inside the terminal to make it easier to change the configuration.

If you want to change the bootcmd, which linux receives from the bootloader, you can find the documentation under <https://www.kernel.org/doc/html/v5.18/admin-guide/kernel-parameters.html>